

DIGITAL STACK

Knowledge. Performance. Growth.

SESIUNEA 2: CONSTRUCȚII PYTHON AVANSATE

AGENDĂ

- List Comprehensions
- Dictionary Comprehensions
- Iteratoare
- Generatori
- Decoratori
- Construirea unui pipeline simplu de procesare date

De la bucle la Python idiomatic

Python oferă modalități concise și expresive pentru procesarea datelor.

În locul unor bucle verbose și a codului repetitiv, Python pune la dispoziție construcții care permit scrierea de cod mai compact, mai lizibil și mai ușor de întreținut.

În această sesiune vom explora câteva dintre cele mai utilizate mecanisme din proiectele Python moderne:

- comprehensions;
- generatori;
- decoratori.

List Comprehensions

List comprehensions reprezintă o metodă concisă de construire a listelor pornind de la o colecție existentă.

Acestea permit transformarea și filtrarea datelor într-o singură expresie.

Avantaje

- cod mai scurt;
- expresivitate crescută;
- utilizare frecventă în proiectele de procesare date și AI.

List Comprehensions - Filtrare

Ce este un modul Python?

Pe lângă transformare, list comprehensions permit și filtrarea elementelor. Această abordare este preferată atunci când dorim să construim o nouă colecție pe baza unei condiții.

Utilizări frecvente

- eliminarea valorilor invalide;
- selecția datelor relevante;
- preprocesarea datelor înainte de analiză.

Dictionary Comprehensions

Dictionary comprehensions permit construirea rapidă a dicționarelor pornind de la alte colecții.

Sunt utile atunci când dorim:

- să asociem chei cu valori calculate;
- să construim statistici;
- să transformăm rezultate într-o structură ușor de interogat.

Avantaje

- reducerea codului repetitiv;
- lizibilitate mai bună;
- exprimarea clară a intenției.

Când folosim comprehensions?

Folosiți comprehensions atunci când:

- construiți o nouă colecție;
- aplicați aceeași transformare tuturor elementelor;
- filtrați elemente pe baza unei condiții simple.

Evitați comprehensions foarte complexe care devin greu de citit.

Lizibilitatea este mai importantă decât scrierea unei expresii într-o singură linie.

Ce este un iterator?

Un iterator reprezintă un obiect care produce elemente unul câte unul.

În loc să ofere toate valorile simultan, acestea sunt furnizate la cerere.

Beneficii:

- consum redus de memorie;
- procesarea colecțiilor mari;
- control mai bun asupra fluxului de date.

Numeroase obiecte Python sunt iterabile:

- liste;
- tuple;
- dicționare;
- fișiere.

Iterabile vs Iteratoare

Un obiect iterabil poate fi parcurs.

Un iterator este mecanismul care produce efectiv elementele.

Conceptual:

- colecția reprezintă datele;
- iteratorul reprezintă modalitatea de acces la aceste date.

Acest concept stă la baza buclelor for și a multor biblioteci de procesare date.

Generatori

Generatorii sunt o metodă simplă de creare a iteratorilor.

În loc să construim și să păstrăm toate valorile în memorie, acestea sunt generate pe măsură ce sunt necesare.

Beneficii

- consum redus de memorie;
- scalabilitate;
- procesarea fișierelor și fluxurilor mari de date.

În proiectele AI și Data Engineering, generatorii sunt utilizați frecvent pentru procesarea dataset-urilor voluminoase.

Return vs Yield

O funcție obișnuită returnează rezultatul și își încheie execuția.

Un generator produce valori succesive și își păstrează starea între execuții.

Diferența conceptuală:

- return finalizează funcția;
- yield suspendă execuția și permite reluarea ulterioară.

Când folosim generatori?

Generatorii sunt utili atunci când:

- citim fișiere mari;
- procesăm volume mari de date;
- construim pipeline-uri de transformare;
- dorim să evităm încărcarea completă a datelor în memorie.

Principiul principal:

- Produce date doar atunci când sunt necesare.

Ce este un decorator?

Un decorator permite extinderea comportamentului unei funcții fără modificarea implementării acesteia.

Decoratorii sunt utilizați pentru:

- logging;
- măsurarea timpului de execuție;
- validare;
- autentificare;
- monitorizare.

Conceptul este întâlnit frecvent în framework-uri moderne precum Flask, FastAPI sau Django.

Avantajele decoratorilor

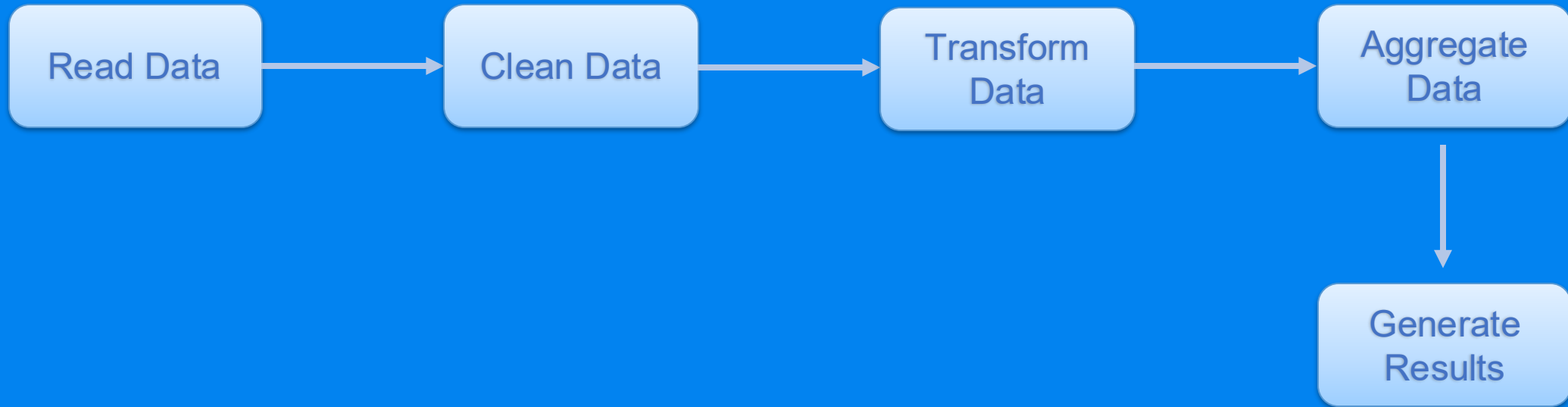
Decoratorii permit separarea responsabilităților.

În loc să adăugăm aceeași logică în mai multe funcții, aceasta poate fi implementată o singură dată și reutilizată.

Beneficii:

- cod mai curat;
- reutilizare;
- întreținere mai ușoară;
- reducerea duplicării.

Aplicarea conceptelor în pipeline



Fiecare etapă poate fi implementată folosind funcții independente.

Exercițiul 1 – Filtrarea și transformarea datelor

Implementați o funcție care primește o listă de numere și returnează o nouă listă care conține exclusiv numerele pare, înmulțite cu 10. Soluția trebuie implementată folosind list comprehension

Exercițiul 2 – Construirea unui dicționar

Implementați o funcție care primește o listă de nume și construiește un dicționar în care cheia reprezintă numele, iar valoarea reprezintă lungimea acestuia.

Soluția trebuie implementată folosind un dictionary comprehension.

Exercițiul 3 – Citirea datelor prin intermediul unui generator

Implementați o funcție care citește un fișier text ce conține câte un nume pe fiecare linie și returnează numele unul câte unul.
Liniile goale nu trebuie incluse în rezultatul final.
Soluția trebuie implementată sub forma unui generator.

Exercițiul 4 – Construirea unui pipeline de procesare date

Utilizând funcțiile dezvoltate anterior, implementați un pipeline care citește numele dintr-un fișier, elimină liniile goale, transformă toate numele în litere mici și calculează numărul de apariții pentru fiecare nume.

În cadrul soluției trebuie utilizate:

- cel puțin un generator;
- cel puțin un list sau dictionary comprehension.

Exercițiul 5 – Monitorizarea execuției

Extindeți pipeline-ul de la exercițiul anterior astfel încât durata totală de execuție să fie afișată automat la rulare.

Implementarea trebuie realizată prin intermediul unui decorator aplicat funcției principale a pipeline-ului.

La finalul execuției trebuie afișat atât rezultatul procesării, cât și timpul de execuție al funcției.

DIGITAL STACK

Knowledge. Performance. Growth.